

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 5**



CONNECT TO THE INTERNET

Oleh:

Regina Silva M

NIM. 2310817220011

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN I
MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to the Internet ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Regina Silva Maharatini

NIM : 2310817220011

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL.....	6
A. Source Code	6
B. Output Program.....	23
C. Pembahasan	23
D. Tautan Git	28

DAFTAR GAMBAR

Gambar 1.Screenshot Tampilan Aplikasi	23
---	----

DAFTAR TABEL

Tabel 1. Source Code Jawaban Soal 1.....	6
Tabel 2. Source Code Jawaban Soal 1.....	7
Tabel 3. Source Code Jawaban Soal 1.....	7
Tabel 4. Source Code Jawaban Soal 1.....	8
Tabel 5. Source Code Jawaban Soal 1.....	10
Tabel 6. Source Code Jawaban Soal 1.....	13
Tabel 7. Source Code Jawaban Soal 1.....	15
Tabel 8. Source Code Jawaban Soal 1.....	18
Tabel 9. Source Code Jawaban Soal 1.....	20
Tabel 10. Source Code Jawaban Soal 1.....	21

SOAL

Soal Praktikum:

1. Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a) Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- b) Gunakan KotlinX Serialization sebagai library JSON.
- c) Gunakan library seperti Coil atau Glide untuk image loading.
- d) API yang digunakan pada modul ini bebas, contoh API gratis The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
- e) Implementasikan konsep data persistence (misalnya offline-first app, pengaturan dark/light mode, fitur favorite, dll)
- f) Gunakan caching strategy pada Room..
- g) Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

1. FlowerRemote.kt

Tabel 1. Source Code Jawaban Soal 1

1	<code>package com.example.modul5.model</code>
2	
3	<code>import kotlinx.serialization.SerialName</code>
4	<code>import kotlinx.serialization.Serializable</code>
5	
6	<code>@Serializable</code>

7	data class FlowerRemote(
8	@SerializedName("name") val name: String,
9	@SerializedName("symbol") val symbol: String,
10	@SerializedName("wikiLink") val wikiLink: String,
11	@SerializedName("imageUrl") val imageUrl: String
12	)

2. FlowerApiService.kt

Tabel 2. Source Code Jawaban Soal 1

1	package com.example.modul5.network
2	
3	import com.example.modul5.model.FlowerRemote
4	import retrofit2.http.GET
5	
6	interface FlowerApiService {
7	@GET("flowers")
8	suspend fun getFlowers(): List<FlowerRemote>
9	}
10	

3. ApiClient.kt

Tabel 3. Source Code Jawaban Soal 1

1	package com.example.modul5.network
2	
3	import
4	com.jakewharton.retrofit2.converter.kotlinx.serialization
5	on.
6	asConverterFactory
7	import kotlinx.serialization.json.Json
8	import okhttp3.MediaType.Companion.toMediaType
9	import retrofit2.Retrofit

10	
11	@OptIn(kotlinx.serialization.ExperimentalSerializationA
12	pi::class)
13	object ApiClient {
14	private const val BASE_URL =
15	"https://birthflowersapi.free.beeceptor.com/"
16	
17	private val json = Json {
18	ignoreUnknownKeys = true
19	}
20	
21	private val retrofit = Retrofit.Builder()
22	.baseUrl(BASE_URL)
23	
24	.addConverterFactory(json.asConverterFactory("applicati
25	on/
26	json".toMediaType()))
27	.build()
28	
29	val apiService: FlowerApiService =
30	retrofit.create(FlowerApiService::class.java)
	}

4. BirthFlowerViewModel.kt

Tabel 4. Source Code Jawaban Soal 1

1	package com.example.modul5
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.viewModelScope
5	import com.example.modul5.model.FlowerRemote
6	import com.example.modul5.network.ApiClient


```

7 import kotlinx.coroutines.flow.MutableStateFlow
8 import kotlinx.coroutines.flow.StateFlow
9 import kotlinx.coroutines.launch
10 import android.util.Log
11
12 class BirthFlowerViewModel : ViewModel() {
13
14     private val _flowers =
15     MutableStateFlow<List<FlowerRemote>>>(emptyList())
16     val flowers: StateFlow<List<FlowerRemote>>> =
17     _flowers
18
19     init {
20         fetchFlowers()
21     }
22
23     private fun fetchFlowers() {
24         viewModelScope.launch {
25             try {
26                 val response =
27                 ApiClient.apiService.getFlowers()
28                 _flowers.value = response
29                 Log.d("BirthFlowerViewModel", "Loaded
30                 ${response.size} flowers")
31             } catch (e: Exception) {
32                 Log.e("BirthFlowerViewModel", "API
33                 Error", e)
34             }
35         }
36     }
37 }

```

5. FlowerListScreen.kt

Tabel 5. Source Code Jawaban Soal 1

1	package com.example.modul5.ui
2	
3	import android.content.Intent
4	import android.net.Uri
5	import androidx.compose.foundation.layout.*
6	import androidx.compose.foundation.lazy.itemsIndexed
7	import androidx.compose.foundation.lazy.LazyColumn
8	import
9	androidx.compose.foundation.shape.RoundedCornerShape
10	import androidx.compose.material3.*
11	import androidx.compose.runtime.Composable
12	import androidx.compose.runtime.collectAsState
13	import androidx.compose.ui.Alignment
14	import androidx.compose.ui.Modifier
15	import androidx.compose.ui.platform.LocalContext
16	import androidx.compose.ui.text.font.FontWeight
17	import androidx.compose.ui.unit.dp
18	import androidx.compose.ui.unit.sp
19	import androidx.lifecycle.viewmodel.compose.viewModel
20	import androidx.navigation.NavController
21	import coil.compose.AsyncImage
22	import coil.request.ImageRequest
23	import com.example.modul5.model.FlowerRemote
24	import com.example.modul5.BirthFlowerViewModel
25	
26	@Composable
27	fun FlowerListScreen(navController: NavController) {
28	val viewModel: BirthFlowerViewModel = viewModel()
29	val flowers = viewModel.flowers.collectAsState()

```

30     val flowerList = flowers.value
31
32     LazyColumn(modifier = Modifier.padding(16.dp)) {
33         itemsIndexed(flowerList) { index: Int, flower:
34 FlowerRemote ->
35         FlowerItem(
36             flower = flower,
37             index = index,
38             navController = navController
39         )
40     }
41 }
42
43
44 @Composable
45 fun FlowerItem(
46     flower: FlowerRemote,
47     index: Int,
48     navController: NavController
49 ) {
50     val context = LocalContext.current
51
52     Card(
53         shape = RoundedCornerShape(8.dp),
54         modifier = Modifier
55             .fillMaxWidth()
56             .padding(8.dp)
57     ) {
58         Row(
59             modifier = Modifier
60                 .padding(16.dp),

```

61	verticalAlignment =
62	Alignment.CenterVertically
63	) {
64	AsyncImage(
65	model = ImageRequest.Builder(context)
66	.data(flower.imageUrl)
67	.crossfade(true)
68	.build(),
69	contentDescription = flower.name,
70	modifier = Modifier.size(100.dp)
71	)
72	
73	Spacer(modifier = Modifier.width(16.dp))
74	
75	Column(modifier = Modifier.weight(1f)) {
76	Text(
77	text = flower.name,
78	style =
79	MaterialTheme.typography.headlineSmall.copy(
80	fontWeight = FontWeight.Bold
81	)
82	)
83	Spacer(modifier = Modifier.height(4.dp))
84	Text(
85	text = flower.symbol,
86	style =
87	MaterialTheme.typography.bodyMedium
88	)
89	Spacer(modifier = Modifier.height(8.dp))
90	
91	Row(

92	horizontalArrangement =
93	Arrangement.spacedBy(8.dp)
94	) {
95	Button(onClick = {
96	val intent =
97	Intent(Intent.ACTION_VIEW, Uri.parse(flower.wikiLink))
98	context.startActivity(intent)
99	}) {
100	Text(text = "Wiki", fontSize =
101	12.sp)
102	}
103	
104	Button(onClick = {
105	
106	navController.navigate("detail/\$index")
107	}) {
108	Text(text = "Detail", fontSize =
109	12.sp)
110	}
111	}
112	}
113	}
114	}
115	}

6. FlowerDetailScreen.kt

Tabel 6. Source Code Jawaban Soal 1

1	package com.example.modul5.ui
2	
3	import androidx.compose.foundation.layout.*
4	import androidx.compose.material3.*

```

5 import androidx.compose.runtime.Composable
6 import androidx.compose.ui.Modifier
7 import androidx.compose.ui.platform.LocalContext
8 import androidx.compose.ui.text.font.FontWeight
9 import androidx.compose.ui.unit.dp
10 import androidx.compose.ui.unit.sp
11 import coil.compose.AsyncImage
12 import coil.request.ImageRequest
13 import com.example.modul5.model.FlowerRemote
14 import androidx.compose.ui.Alignment
15 import androidx.compose.ui.text.style.TextAlign
16
17 @Composable
18 fun FlowerDetailScreen(flower: FlowerRemote) {
19     val context = LocalContext.current
20
21     Column(
22         modifier = Modifier
23             .padding(24.dp)
24             .fillMaxSize(),
25         verticalArrangement = Arrangement.Top,
26         horizontalAlignment =
27 Alignment.CenterHorizontally
28     ) {
29         AsyncImage(
30             model = ImageRequest.Builder(context)
31                 .data(flower.imageUrl)
32                 .crossfade(true)
33                 .build(),
34             contentDescription = flower.name,
35             modifier = Modifier.size(280.dp)

```

36	)
37	
38	Spacer(modifier = Modifier.height(24.dp))
39	
40	Text(
41	text = flower.name,
42	fontSize = 26.sp,
43	fontWeight = FontWeight.Bold
44	)
45	
46	Spacer(modifier = Modifier.height(12.dp))
47	
48	Text(
49	text = flower.symbol,
50	style = MaterialTheme.typography.bodyLarge,
51	textAlign = TextAlign.Center
52	)
53	}
54	}

7. MainActivity.kt

Tabel 7. Source Code Jawaban Soal 1

1	package com.example.modul5
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.foundation.layout.padding
7	import androidx.compose.material.icons.Icons

```

8 import androidx.compose.material.icons.filled.DarkMode
9 import
10 androidx.compose.material.icons.filled.LightMode
11 import androidx.compose.material3.*
12 import androidx.compose.runtime.collectAsState
13 import androidx.compose.ui.Modifier
14 import androidx.lifecycle.viewmodel.compose.viewModel
15 import androidx.navigation.compose.NavHost
16 import androidx.navigation.compose.composable
17 import
18 androidx.navigation.compose.rememberNavController
19 import com.example.modul5.ui.FlowerDetailScreen
20 import com.example.modul5.ui.FlowerListScreen
21 import
22 com.example.modul5.ui.theme.BirthFlowerzAppTheme
23
24 @OptIn(ExperimentalMaterial3Api::class)
25 class MainActivity : ComponentActivity() {
26     override fun onCreate(savedInstanceState: Bundle?)
27     {
28         super.onCreate(savedInstanceState)
29         setContent {
30             val themeViewModel: ThemeViewModel =
31             viewModel()
32             val isDark =
33             themeViewModel.isDarkMode.collectAsState()
34
35             BirthFlowerzAppTheme(darkTheme =
36             isDark.value) {
37                 val navController =
38                 rememberNavController()

```



```

39
40         Scaffold(
41             topBar = {
42                 TopAppBar(
43                     title = { Text("Birth
44 Flowers") },
45                     actions = {
46                         IconButton(onClick = {
47 themeViewModel.toggleTheme() }) {
48                             Icon(
49                                 imageVector =
50 if (isDark.value) Icons.Filled.DarkMode else
51 Icons.Filled.LightMode,
52
53 contentDescription = "Toggle Theme"
54                             )
55                         }
56                     }
57                 )
58             }
59         ) { innerPadding ->
60             NavHost(
61                 navController = navController,
62                 startDestination =
63 "main_screen",
64                 modifier =
65 Modifier.padding(innerPadding)
66             ) {
67                 composable("main_screen") {
68
69 FlowerListScreen(navController = navController)

```

70	}
71	
72	composable("detail/{flowerIndex}") { backStackEntry ->
73	val index =
74	backStackEntry.arguments?.getString("flowerIndex")?.to
75	IntOrNull()
76	val flowerViewModel:
77	BirthFlowerViewModel = viewModel()
78	val flowers =
79	flowerViewModel.flowers.collectAsState().value
80	val flower = index?.let {
81	flowers.getOrNull(it) }
82	flower?.let {
83	
84	FlowerDetailScreen(flower = it)
85	}
86	}
87	}
88	}
89	}
90	}
91	}
92	}

8. ThemeViewModel.kt

Tabel 8. Source Code Jawaban Soal 1

1	package com.example.modul5
2	
3	import android.app.Application

```

4 import androidx.lifecycle.AndroidViewModel
5 import androidx.lifecycle.viewModelScope
6 import com.example.modul5.datastore.ThemePreference
7 import kotlinx.coroutines.flow.MutableStateFlow
8 import kotlinx.coroutines.flow.StateFlow
9 import kotlinx.coroutines.flow.first
10 import kotlinx.coroutines.launch
11
12 class ThemeViewModel(application: Application) :
13     AndroidViewModel(application) {
14
15         private val _isDarkMode = MutableStateFlow(false)
16         val isDarkMode: StateFlow<Boolean> get() =
17             _isDarkMode
18
19         init {
20             viewModelScope.launch {
21                 val current =
22                     ThemePreference.getThemeSetting(getApplication()).first()
23                 _isDarkMode.value = current
24             }
25         }
26
27         fun toggleTheme() {
28             val context = getApplication<Application>()
29             viewModelScope.launch {
30                 val newValue = !_isDarkMode.value
31                 ThemePreference.saveThemeSetting(context,
32                     newValue)
33                 _isDarkMode.value = newValue
34             }
35         }
36     }

```

35	}
36	}
37	}
38	

9. ThemePreference.kt

Tabel 9. Source Code Jawaban Soal 1

1	package com.example.modul5.datastore
2	
3	import android.content.Context
4	import
5	androidx.datastore.preferences.core.booleanPreferencesKey
6	import androidx.datastore.preferences.core.edit
7	import
8	androidx.datastore.preferences.preferencesDataStore
9	import kotlinx.coroutines.flow.Flow
10	import kotlinx.coroutines.flow.map
11	
12	private const val DATASTORE_NAME = "settings"
13	val Context.dataStore by preferencesDataStore(name =
14	DATASTORE_NAME)
15	
16	object ThemePreference {
17	private val DARK_MODE_KEY =
18	booleanPreferencesKey("dark_mode")
19	
20	fun getThemeSetting(context: Context): Flow<Boolean>
21	{
22	return context.dataStore.data.map { preferences -

```

23 >
24     preferences[DARK_MODE_KEY] ?: false
25     }
26 }
27
28 suspend fun saveThemeSetting(context: Context,
29 isDarkMode: Boolean) {
30     context.dataStore.edit { preferences ->
31         preferences[DARK_MODE_KEY] = isDarkMode
32     }
33 }
34 }

```

10. Theme.kt

Tabel 10. Source Code Jawaban Soal 1

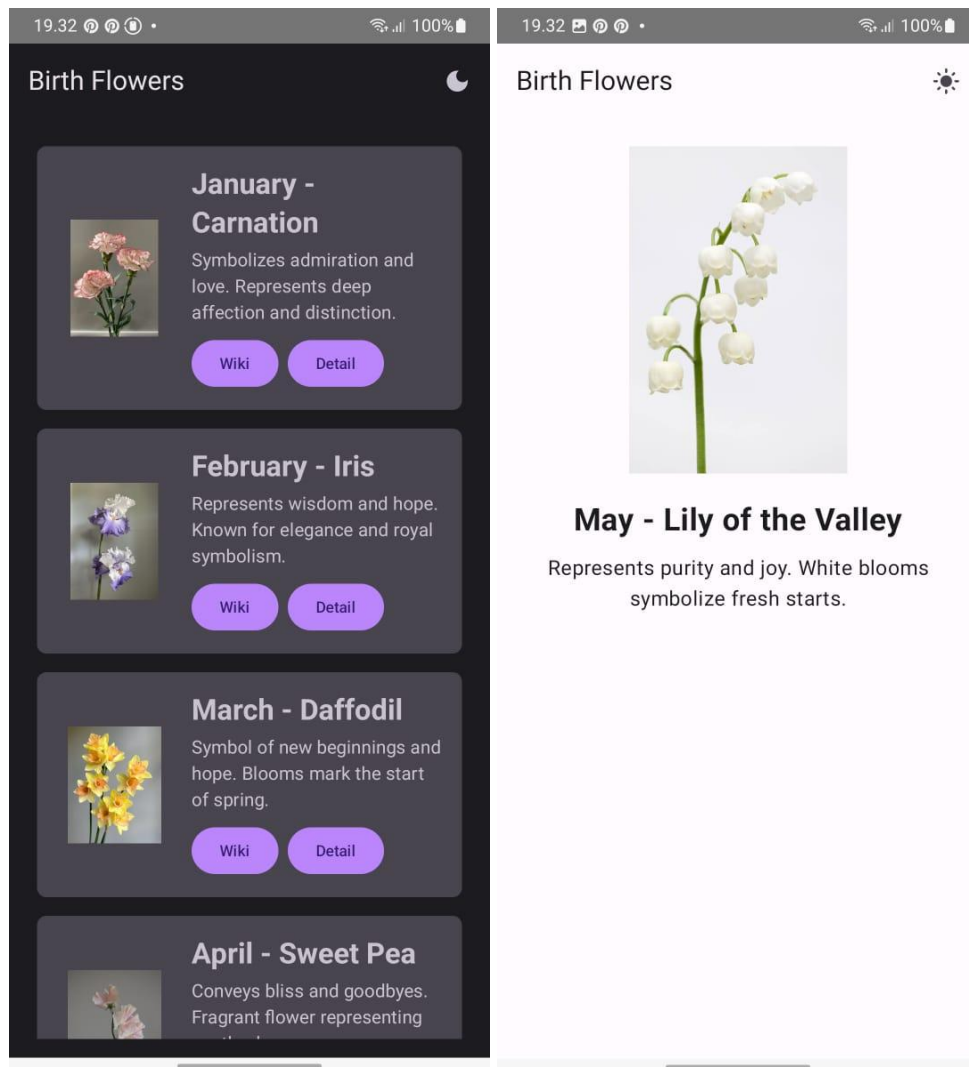
```

1 package com.example.modul5.ui.theme
2
3 import androidx.compose.material3.MaterialTheme
4 import androidx.compose.material3.darkColorScheme
5 import androidx.compose.material3.lightColorScheme
6 import androidx.compose.runtime.Composable
7 import androidx.compose.material3.Typography
8 import androidx.compose.material3.ColorScheme
9
10 private val DarkColorScheme = darkColorScheme(
11     primary =
12     androidx.compose.ui.graphics.Color(0xFFBB86FC),
13     secondary =
14     androidx.compose.ui.graphics.Color(0xFF03DAC6)

```

```
15 )
16
17 private val LightColorScheme = lightColorScheme(
18     primary =
19     androidx.compose.ui.graphics.Color(0xFF6200EE),
20     secondary =
21     androidx.compose.ui.graphics.Color(0xFF03DAC6)
22 )
23
24 @Composable
25 fun BirthFlowerzAppTheme(
26     darkTheme: Boolean = false,
27     content: @Composable () -> Unit
28 ) {
29     val colors: ColorScheme = if (darkTheme)
30     DarkColorScheme else LightColorScheme
31
32     MaterialTheme(
33         colorScheme = colors,
34         typography = Typography(),
35         content = content
36     )
37 }
38
39
```

B. Output Program



Gambar 1. Screenshot Tampilan Aplikasi

C. Pembahasan

1. FlowerRemote.kt

File ini berisi *data class* yang digunakan untuk menampung data dari API.

Anotasi `@Serializable` dan `@SerializedName` digunakan dari library `kotlinx.serialization` agar JSON dari server bisa langsung dipetakan ke objek Kotlin.

- name: nama bunga
- symbol: makna atau simbolis dari bunga tersebut

- `wikiLink`: link ke Wikipedia untuk informasi lengkap
 - `imageUrl`: alamat gambar bunga
- File ini penting karena menjadi jembatan antara data dari internet dan tampilan aplikasi.

2. `FlowerApiService.kt`

File ini adalah interface Retrofit yang mendefinisikan endpoint API.

Fungsi `getFlowers()` menggunakan anotasi `@GET("flowers")` untuk mengambil data dari URL `https://birthflowersapi.free.beeceptor.com/flowers`.

Kata `suspend` menunjukkan bahwa fungsi ini berjalan secara asynchronous dengan `coroutine`. Hasilnya berupa list dari `FlowerRemote`, yaitu data bunga yang akan ditampilkan ke layar. Singkatnya, file ini mendefinisikan *apa* yang mau diambil dari internet.

3. `ApiClient.kt`

File ini berfungsi untuk menyiapkan Retrofit, yaitu library yang digunakan untuk komunikasi internet.

- `BASE_URL` adalah alamat dasar dari API.
 - `Json { ignoreUnknownKeys = true }` memastikan kalau JSON yang tidak dikenali tidak bikin error.
 - `asConverterFactory(...)` memungkinkan penggunaan `kotlinx.serialization` sebagai converter JSON.
 - `retrofit.create(FlowerApiService::class.java)` membuat objek `apiService` yang nantinya bisa dipakai di `ViewModel`.
- File ini hanya dibuat sekali dan dipakai di mana saja saat ingin ambil data dari internet.

4. `BirthFlowerViewModel.kt`

File ini berfungsi sebagai pengelola data utama aplikasi. `BirthFlowerViewModel` adalah kelas turunan dari `ViewModel` yang menggunakan `StateFlow` untuk menyimpan dan mengalirkan data daftar bunga secara reaktif. Di dalamnya, fungsi `fetchFlowers()` dipanggil saat `ViewModel` pertama kali dibuat melalui `init`, dan digunakan untuk mengambil data dari API menggunakan `Retrofit` yang sudah disiapkan di `ApiClient`. Fungsi ini berjalan dalam `viewModelScope.launch`, artinya berjalan secara asynchronous agar tidak membebani UI. Jika sukses, datanya dimasukkan ke `_flowers`, dan bila gagal akan muncul log error di `Logcat`. Pendekatan ini juga memastikan data tetap aman saat orientasi layar berubah, karena `ViewModel` tidak ikut dihancurkan seperti `Activity`.

5. FlowerListScreen.kt

File ini menampilkan daftar bunga dalam bentuk scrollable list menggunakan `LazyColumn` dari `Jetpack Compose`. Data bunga diambil dari `BirthFlowerViewModel` dengan `collectAsState()` sehingga tampilannya akan otomatis ter-update saat data berubah. Di dalam list, setiap item ditampilkan menggunakan komponen `FlowerItem`. Komponen ini terdiri dari gambar bunga, nama, dan simbolnya, serta dua tombol. Tombol pertama membuka halaman Wikipedia dari bunga tersebut menggunakan `Intent`, sedangkan tombol kedua akan mengarahkan pengguna ke halaman detail dengan mengirimkan index item ke `NavController`. File ini menerapkan prinsip pemisahan logika dan UI dengan baik, karena semua logika data disimpan di `ViewModel`, sementara file ini hanya fokus pada tampilan dan interaksi pengguna.

6. FlowerDetailScreen.kt

File ini menampilkan halaman detail untuk satu bunga yang dipilih dari list. Komponen ini menerima parameter `flower` yang berisi data lengkap dari `FlowerRemote`, seperti nama, gambar, dan simbol. Layout-nya tersusun secara vertikal dengan `Column`, dan gambar bunga ditampilkan menggunakan `AsyncImage` dari `Coil`, yang langsung memuat gambar dari internet. Di bawah gambar, ditampilkan nama bunga dengan ukuran font besar dan tebal, lalu simbol bunga dengan gaya body text dan rata tengah. Tampilan ini sederhana

namun efektif untuk memberikan informasi detail bunga kepada pengguna setelah mereka menekan tombol "Detail" dari halaman sebelumnya.

7. MainActivity.kt

File `MainActivity.kt` adalah titik masuk utama aplikasi saat dijalankan. Di dalam metode `onCreate()`, aplikasi menggunakan `setContent` untuk menampilkan UI berbasis Jetpack Compose. Hal pertama yang dilakukan adalah mengambil instance dari `ThemeViewModel`, lalu membaca nilai `isDarkMode` yang menunjukkan apakah tema saat ini gelap atau terang. Komponen `BirthFlowerzAppTheme` kemudian digunakan untuk menerapkan tema tersebut ke seluruh aplikasi. Navigasi antar layar diatur menggunakan `NavHost`, dengan dua layar utama: `main_screen` yang menampilkan daftar bunga, dan `detail/{flowerIndex}` yang menampilkan detail bunga berdasarkan indeks. Fitur dark mode dikontrol melalui `TopAppBar`, yang memiliki tombol toggle dengan ikon `DarkMode` dan `LightMode`. Saat pengguna menekan tombol tersebut, fungsi `toggleTheme()` akan dijalankan untuk mengubah tema secara real-time dan menyimpannya.

8. ThemeViewModel.kt

`ThemeViewModel.kt` adalah `ViewModel` khusus untuk mengelola preferensi tema aplikasi (gelap atau terang). Kelas ini mewarisi `AndroidViewModel` agar bisa mengakses konteks aplikasi secara langsung. Di dalamnya terdapat `MutableStateFlow` yang menyimpan status dark mode dan dapat diamati secara real-time oleh komponen UI. Saat `ViewModel` dibuat (`init`), ia langsung membaca nilai yang disimpan di `DataStore` menggunakan fungsi `getThemeSetting()`, lalu memperbarui nilai `_isDarkMode`. Fungsi `toggleTheme()` digunakan untuk membalikkan nilai tema saat ini, menyimpannya kembali ke `DataStore`, dan memperbarui nilai yang diamati oleh UI. Dengan pendekatan ini, perubahan tema dapat dipertahankan bahkan setelah aplikasi ditutup dan dibuka kembali.

9. ThemePreference.kt

File ini berisi objek `ThemePreference`, yang bertanggung jawab menyimpan dan mengambil pengaturan dark mode menggunakan Jetpack `DataStore`. Data disimpan dalam bentuk boolean di dalam file `settings.preferences_pb`. Dengan menggunakan key `DARK_MODE_KEY`, fungsi `getThemeSetting()` mengembalikan `Flow<Boolean>` yang terus memantau perubahan preferensi, sementara fungsi `saveThemeSetting()` digunakan untuk menyimpan nilai baru ke `DataStore`. Karena operasi ini berjalan asynchronous, `suspend` digunakan agar bisa dipanggil dari coroutine. `Context.dataStore` juga dibuat sebagai ekstensi agar mudah diakses dari kelas mana pun yang memiliki konteks Android.

10. Theme.kt

File ini mendefinisikan dua skema warna utama aplikasi: satu untuk mode terang dan satu lagi untuk mode gelap. Warna-warna ini ditentukan menggunakan `darkColorScheme` dan `lightColorScheme` dari Material 3. Fungsi `BirthFlowerzAppTheme()` menerima parameter `darkTheme` untuk memilih skema warna yang akan digunakan, lalu menerapkan skema dan tipografi melalui `MaterialTheme`. Fungsi ini membungkus seluruh UI aplikasi, memastikan semua komponen mengikuti gaya warna yang konsisten. Dengan pendekatan ini, pengguna dapat melihat perubahan warna langsung saat beralih dari light ke dark mode atau sebaliknya, dengan pengalaman visual yang mulus.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/rgnsilvas/Pemrograman-Mobile/tree/main/MODUL5>